

ADR-001 · Run Langflow via a local uv/pip install instead of Docker

Field	Detail
Decision	Abandon the Docker Desktop route and run Langflow from a local Python virtual environment.
Date	2026-07-23
Context	The Docker path hit two consecutive blockers on this Windows machine: a Docker Hub DNS resolution failure inside WSL2 (BUG-001) and an immediate container exit because the image now requires superuser credentials (BUG-002). Both were solvable, but added friction for a single-user local learning setup.
Options Considered	A. Docker — official one-liner, isolated, but fought WSL2 DNS state and mandatory-auth startup. · B. Local uv venv + pip — no daemon, direct <code>langflow run</code> , closer to the code, but exposed to Windows wheel/build issues. · C. Langflow Desktop installer — avoids Docker and Python entirely, but least transparent for a learning project.
Chosen Solution	Local install: <code>uv venv .venv --python 3.12</code> , activate, <code>uv pip install --only-binary=litellm "langflow==1.10.2"</code> , then <code>langflow run</code> → <code>http://127.0.0.1:7860</code> .
Rationale	A learning project benefits from the most transparent, quickest-to-iterate setup. Removing the Docker/WSL2 layer eliminated two whole classes of failure (BUG-001, BUG-002) at the cost of one Windows packaging issue (BUG-005) that turned out to be pinnable.
Impact	Establishes the <code>.venv</code> + <code>langflow run</code> workflow used throughout the project. Directly leads to ADR-002 (version pin) and ADR-003 (Python version). Supersedes the Docker approach; BUG-001/BUG-002 are closed as no-longer-on-the-path.

ADR-002 · Pin langflow==1.10.2 with --only-binary=litellm instead of latest (1.11.0)

Field	Detail
Decision	Install langflow==1.10.2 and force binary-only resolution of litellm, rather than installing the latest langflow 1.11.0.
Date	2026-07-23
Context	langflow 1.11.0 transitively requires litellm==1.93.0, which has no Windows wheel and fails to compile from source without the MSVC linker (BUG-005).
Options Considered	A. Install MSVC C++ Build Tools and build litellm 1.93.0 from source for 1.11.0 — unblocks the latest version but is a multi-GB toolchain install for one native dependency. · B. Pin langflow==1.10.2 — resolves litellm==1.91.4, which ships a win_amd64 wheel and needs no compiler. · C. Fall back to Docker — reintroduces BUG-001/BUG-002.
Chosen Solution	uv pip install --only-binary=litellm "langflow==1.10.2" (resolves litellm==1.91.4); --only-binary=litellm makes pip fail loudly rather than silently attempting a source build.
Rationale	Fastest path to a working Langflow with zero native-build risk. 1.10.2 officially supports Python 3.12, and the learning tasks (basic prompting, loops, a simple agent) do not need 1.11.0-specific features.
Impact	Fixes BUG-005. Pins the runtime Langflow version documented in Architecture.md. Can be revisited if a 1.11.x feature is needed (then option A applies).

ADR-003 · Use Python 3.12 for the virtual environment instead of 3.14

Field	Detail
Decision	Create the project <code>.venv</code> on CPython 3.12, not the machine-default 3.14.
Date	2026-07-23
Context	The system default is Python 3.14.2. A first <code>.venv</code> was accidentally built on 3.14, and several Langflow dependencies lack 3.14 wheels, pushing them toward source builds.
Options Considered	A. Python 3.14.2 (system default) — newest, but too new; missing wheels force compilation. · B. Python 3.12.10 (also installed) — officially supported by Langflow 1.10.2, broad wheel coverage.
Chosen Solution	<code>uv venv --python 3.12 --clear .venv</code> (the <code>--clear</code> recreates the mistakenly-3.14 <code>venv</code>), then verify with <code>python -V</code> showing 3.12.x before installing.
Rationale	Langflow 1.10.2 officially targets 3.12, and 3.12 has mature wheels across the dependency tree, avoiding avoidable source builds. (Note: 3.12 did not by itself fix <code>litellm</code> — that was a packaging gap regardless of interpreter; see BUG-005.)
Impact	Defines the interpreter for <code>.venv</code> . Prerequisite for ADR-002's clean install.

ADR-004 · Use Groq (llama-3.1-8b-instant) as the flow LLM provider

Field	Detail
Decision	Wire the flows to Langflow's Groq component running llama-3.1-8b-instant rather than an OpenAI model.
Date	2026-07-23
Context	The flows needed a chat LLM. Two of the flow files carry the stock template description "Perform basic prompting with an OpenAI model", but the actual model node in every prompting flow is a GroqModel.
Options Considered	A. OpenAI — the template default, requires an OpenAI key. · B. Groq — fast inference, free-tier friendly for a learning project, drop-in via the built-in Groq component.
Chosen Solution	GroqModel component with model_name = llama-3.1-8b-instant, used in Basic LLM Prompting.json and the cyclic Basic Prompting*.json flows.
Rationale	Groq gives low-latency, low-cost iteration suitable for experimentation. The leftover "OpenAI" wording is a template artifact, not the running config.
Impact	The flow files' description fields are inaccurate (say OpenAI; actually Groq) — flagged in Architecture.md and Research topic 3. Affects flows/Basic LLM Prompting.json, flows/Basic Prompting.json, flows/Basic Prompting (1).json.

ADR-005 · Test cyclic support with the built-in Loop component, not a raw LLM feedback loop

Field	Detail
Decision	Verify Langflow's cyclic-execution support using the dedicated Loop component over a bounded data list, rather than manually wiring an LLM's output back to its own input.
Date	2026-07-23
Context	The goal was to confirm Langflow accepts a real back-edge and re-executes a section. A naive cycle (LLM output → LLM input) can run away and keep making API calls if the stop condition fails.
Options Considered	A. Loop component — explicit back-edge (Item → ... → Looping) with a guaranteed stop when the input list is exhausted; aggregates via Done . · B. Unrestricted LLM feedback loop — tests a cycle but risks runaway Groq calls with no reliable termination.
Chosen Solution	Read File (dataframe) → Loop; Loop.item → Type Convert → Prompt Template (task) ← Chat Input (user_input) → Groq → Type Convert (JSON) → Loop.Looping (back-edge); Loop.done → Type Convert → Chat Output. Fed by data/langflow_cycle_test.csv (3 rows).
Rationale	The Loop component gives a controlled, terminating cycle — enough to prove the back-edge is accepted and a section re-executes per row, without unbounded API spend.
Impact	Realized in flows/Basic Prompting.json / Basic Prompting (1).json. Produced the findings in Research topics 1 and 2 (cyclic support confirmed; per-iteration interactive input is <i>not</i> supported by this pattern).

ADR-006 · Port the "Memora" RAG architecture into Langflow as one generated, Custom-Component flow

Field	Detail
Decision	Rebuild an external LangGraph-based agentic RAG system ("Memora": dual-track retrieval, NAC/DC/LBC context compression, four LLM-judge validators, MongoDB feedback/thumbdowns, self-learning distillation, a FIFO rate-limit gate) as a single Langflow flow, with every Memora-specific behavior implemented as Python Custom Components generated by a script rather than hand-built in the UI.
Date	2026-07-27
Context	Langflow's built-in nodes cover generic RAG plumbing (loaders, splitters, embeddings, vector stores, LLM calls) but have no equivalent for Memora's judge/compression/routing logic; that logic had to be ported as raw Python regardless of approach.
Options Considered	A. Native nodes only — infeasible, no built-in node covers judges, compression, or the retry/routing logic. B. Hand-build ~20+ Custom Components directly in the Langflow UI , mirroring each LangGraph node one at a time. C. Generate the Custom Component source and the flow JSON from a script , driven directly by the original Python node files, and validate with <code>Graph.from_payload</code> plus a fake-LLM/embeddings test harness before ever opening the UI.
Chosen Solution	C.
Rationale	A generated flow can be round-tripped and unit-tested headlessly. This caught 13 real behavioral divergences from the source node files (wrong retry-count semantics, wrong chunk shape, missing pre-retrieval dedup, validators that mutated state when the source only logs, etc.) that would have been invisible from clicking through the UI.
Impact	Created <code>flows/LangGraph RAG Pipeline.json</code> (grew from ~39 to 47+ components across the session as ingestion, per-role LLM clients, and native Knowledge/Memory Base nodes were added — see ADR-007, ADR-008). Establishes the working pattern for any further parity work: edit the generator, re-run the test suite, re-export — not hand-edit the exported JSON.

ADR-007 · Use Langflow's Knowledge node for retrieval instead of a hand-wired ChromaDB client

Field	Detail
Decision	Replace the ported <code>chromadb.PersistentClient</code> -based retrieval component with two native Langflow <code>Knowledge</code> nodes in Retrieve mode (<code>documents</code> , <code>learned_qa</code>) feeding a thin adapter component.
Date	2026-07-27
Context	Explicit request to keep the RAG logic but swap the persistence layer for Langflow-native equivalents wherever one exists, rather than a bespoke Python vector-store client.
Options Considered	A. Keep the hand-rolled Chroma client — full control over query parameters and thresholds, but bypasses Langflow's own Knowledge Base management (creation, embedding-model choice, backend selection) entirely. B. Knowledge node — managed inside Langflow's UI/DB, but is a per-user resource a flow export cannot carry (BUG-007), and Langflow fans out on graph edges rather than on data, so it can't issue one retrieval call per query variant the way the original <code>Send()</code> -based fan-out did.
Chosen Solution	B.
Rationale	Matches the explicit ask, and keeps the Knowledge Base lifecycle (ingestion, embedding model, backend) inside Langflow's own tooling instead of duplicating it in project code.
Impact	<code>retrieve</code> became a pure adapter converting <code>Knowledge</code> result rows into the flat chunk shape the rest of the pipeline expects; <code>generate_query_variants</code> gained a <code>Search Query</code> output selecting which variant is searched on each retry pass, to compensate for the loss of per-variant fan-out; per-track similarity floors moved into <code>retrieve</code> . Directly exposed the distance-vs-similarity mismatch fixed in BUG-010 / Research topic 6.

ADR-008 · Drop the Memory Base node rather than require one per import

Field	Detail
Decision	Remove the <code>Memory Base</code> node (added to replace the MongoDB feedback/thumbdown store) from the flow entirely, instead of documenting "create and attach a Memory Base" as a required per-import setup step.
Date	2026-07-27
Context	Like <code>Knowledge</code> , a <code>Memory Base</code> is a per-user, per-workflow-attached resource that a flow export cannot carry (BUG-007); every fresh import failed the pre-flight check until one was created and selected.
Options Considered	A. Keep it , document the manual per-import step. B. Remove it , accept that <code>blocked_variants</code> / <code>prior_thumbdowns</code> reset every run instead of persisting across sessions.
Chosen Solution	B — explicit user choice when offered the trade-off.
Rationale	Removes a recurring import-time blocker for a feature (<code>user_input</code> / <code>cmd_*</code> already treat memory as an optional input) that Langflow can't make importable anyway.
Impact	Cross-run learning from prior thumbdowns/failed variants is currently unavailable; within a single run the retry cycle still tracks blocked variants in-memory. Re-adding is a single generator flag (<code>ENABLE_MEMORY_BASE</code>) away, not a redesign.

ADR-009 · Collapse the retry cycle into one deterministic answer path

Field	Detail
Decision	Remove the two back-edges (<code>combine_tracks.to_retry</code> and <code>check_answer_quality.to_retry</code> → <code>generate_query_variants</code>) and flatten the routing so exactly one path reaches <code>generate_answer</code> : <code>combine_tracks.to_draft</code> → <code>generate_draft</code> → <code>check_answer_quality.to_answer</code> → <code>generate_answer</code> .
Date	2026-07-27
Context	The multi-branch conditional routing (mirroring the original LangGraph conditional edges) produced <code>generate_draft</code> has not been built yet — Langflow read an inactive branch's output before the active branch had populated the shared input field (BUG-010, Research topic 7). This change was made in a separate assistant session continuing the same exported flow file; recorded here for traceability since it changes the flow's behavior materially.
Options Considered	A. Keep full conditional routing , root-cause the build-order interaction between competing conditional outputs feeding one field. B. Flatten to a single deterministic path , trading away the original retry-on-bad-grounding / retry-on-empty-context behavior for a flow that reliably produces an answer.
Chosen Solution	B.
Rationale	Unblocked the flow from ever reaching an answer at all, which took priority over preserving the retry behavior mid-debugging session.
Impact	The flow became acyclic at 47 components plus 4 nodes / 85 edges (the current export remains 51 total nodes / 85 edges). <code>generate_query_variants</code> no longer re-runs on a low-grounding verdict or empty retrieval — this is a real regression against the ported Memora design (see BUG-010) and should be revisited once the underlying build-order hazard is understood, rather than left as the permanent shape of the flow.

ADR-010 · Use the local OpenAI-compatible server for the RAG pipeline

Field	Detail
Decision	Replace the RAG pipeline's three Groq clients with <code>ChatOpenAI</code> clients pointed at <code>http://192.168.1.13:3001/v1</code> , retaining <code>llama-3.1-8b-instant</code> for the Generation, Judge, and JSON Repair roles.
Date	2026-07-27
Context	The model is now hosted on a LAN server rather than Groq. The server exposes an OpenAI-compatible chat-completions API, and the user created a Langflow global variable named <code>LOCAL-LLM</code> for its API key.
Options Considered	A. Keep the built-in Groq components and override only <code>base_url</code> — retains Groq-specific discovery and dependencies. B. Use <code>ChatOpenAI</code> against the local endpoint — matches the server protocol and removes Groq runtime/API coupling.
Chosen Solution	B. The export keeps the existing edge-compatible component identity internally, but its runtime implementation is <code>langchain_openai.ChatOpenAI</code> ; no Groq URL, client, or embedded key is used.
Rationale	OpenAI-compatible clients are the correct abstraction for a local <code>/v1/chat/completions</code> server. The same three role-specific temperatures and output-token limits remain intact.
Impact	Each fresh import must reselect the per-user <code>LOCAL-LLM</code> global variable if Langflow does not preserve the database-backed binding. The API base must remain <code>/v1</code> because the client appends <code>/chat/completions</code> . <code>/v1/models</code> returned 404 during validation, so model discovery is not relied upon and the model ID is explicit. ADR-004 remains valid for the older basic prompting flows only.